Jio Platforms Limited

NGINX LUA GUIDE

Consent Management System

COPYRIGHT INFORMATION
No part of this document may be copied, reproduced, stored in any retrieval system, or transmitted in any form or by any means, either electronically, mechanically, or otherwise, without prior written permission.

© Copyright Jio Platforms Limited.
All rights reserved.
Jio Platforms Limited, Reliance Corporate Park
Thane Belapur Road, Ghansoli-400 701

TRADEMARKS
All the brand names and other products or services mentioned in this document are identified by the trademarks or service marks of their respective owners

DISCLAIMER
The information in this document is subject to change without notice and should not be construed as a commitment by Jio Platforms Limited. assumes no responsibility or makes no warranties for any errors that may appear in this document and disclaims any implied warranty of merchantability or fitness for a particular purpose

Date: November 26, 2025 *Released document | Version 1.0*

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Version History

Date	Contributor	Contribution	Version	State
Nov 25, 2025	Racharla Vamshi	Initial Draft	0.1	Draft
Nov 26, 2025	Lokesh Sharma	Reviewed and released the document	1.0	Released

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

Table of Contents

Version History *1*

1. Download and extract the OpenResty source: *3*

2. Create a portable directory structure: *3*

3. Copy LuaJIT libraries and headers into the portable directory if these files are not there install them using `sudo yum install luajit luajit-devel`: *3*

4. Configure OpenResty to use the portable LuaJIT paths: *3*

5. Build and install: *3*

6. Verify the portable NGINX build: *3*

7. Kill any existing nginx instances if needed: *3*

8. Start the portable NGINX: *4*

9. Package the portable build: *4*

Configuration: *4*

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

1. Download and extract the OpenResty source:

- wget https://openresty.org/download/openresty-VERSION.tar.gz
  tar -xzf openresty-VERSION.tar.gz
  cd openresty-VERSION
- recommended version & tested : 1.25.3.2

2. Create a portable directory structure:

- mkdir -p /app/lua/luajit/lib
- mkdir -p /app/lua/luajit/include

3. Copy LuaJIT libraries and headers into the portable directory if these files are not there install them using `sudo yum install luajit luajit-devel`:

```
cp /usr/local/lib/libluajit-5.1.so* /app/lua/luajit/lib/
cp -r /usr/local/include/luajit-2.1/* /app/lua/luajit/include/
```

4. Configure OpenResty to use the portable LuaJIT paths:

```
./configure \
  --prefix=/app/lua \
  --with-http_ssl_module \
  --with-http_v2_module \
  --with-http_stub_status_module \
  --with-pcre-jit \
  --with-luajit \
  --with-cc-opt="-I/app/lua/luajit/include" \
  --with-ld-opt="-L/app/lua/luajit/lib -Wl,-rpath,/app/lua/luajit/lib"
```

5. Build and install:

```
make
make install
```

6. Verify the portable NGINX build:

```
cd /app/lua/nginx/sbin
./nginx -t
```

7. Kill any existing nginx instances if needed:

```
ps -ef | grep nginx
kill -9 <pid>
```

"CONFIDENTIAL – FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

8. Start the portable NGINX:

./nginx

9. Package the portable build:

cd /app
zip -r nginx-lua.zip lua
The output nginx-lua.zip is a fully portable build that runs on any Linux machine.

Configuration:

This almost looks like normal nginx configuration but has some extra configuration like.
example:

```
server {
    listen 8443 ssl;
    server_name api.negd.com;

    ssl_certificate     /app/middleware/nginx/cert/jcms.negd.pem;
    ssl_certificate_key
/app/middleware/nginx/cert/jcms.negd.com.key;

    server_tokens off;

    ssl_session_cache shared:SSL:1m;
    ssl_session_timeout 10m;
    ssl_protocols TLSv1.2;
    ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-
SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-
POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
    ssl_prefer_server_ciphers on;

    access_log /app/lua/nginx/logs/ip_debug.log ipdebug;

    location /pizzashack/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/sessiontest.lua;
        proxy_pass https://jcms.api.gateway/pizzashack/;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
    }

    location /test {
        default_type text/plain;
```

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
        content_by_lua_block {
            ngx.say("unsecured works")
        }
    }

    location /oauth2/ {
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        proxy_read_timeout 5m;
        proxy_send_timeout 5m;
        proxy_hide_header X-Forwarded-Host;
        proxy_hide_header X-Forwarded-Server;
        proxy_hide_header X-Forwarded-For;
        proxy_hide_header Referrer-Policy;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-XSS-Protection;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;

        proxy_pass http://jcms.auth.com/oauth2/;
        allow 127.0.0.1;
        deny all;
    }

    location /secureCode/ {
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        proxy_read_timeout 5m;
        proxy_send_timeout 5m;
        proxy_hide_header X-Forwarded-Host;
        proxy_hide_header X-Forwarded-Server;
        proxy_hide_header X-Forwarded-For;
        proxy_hide_header Referrer-Policy;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-XSS-Protection;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;

        proxy_pass http://jcms.auth.com/secureCode/;
        allow 127.0.0.1;
        deny all;
```

"CONFIDENTIAL - FOR NeGD(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
    }

    location /api/securecode/ {

        if ($request_method = OPTIONS) {
            add_header 'Access-Control-Allow-Origin' '*';
            add_header 'Access-Control-Allow-Methods' 'GET, POST,
PUT, PATCH, DELETE, OPTIONS';
            add_header 'Access-Control-Allow-Headers' '*';
            add_header 'Access-Control-Max-Age' 86400;
            add_header 'Content-Length' 0;
            add_header 'Content-Type' 'text/plain';
            return 204;
        }

        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;
        proxy_read_timeout 5m;
        proxy_send_timeout 5m;
        proxy_hide_header X-Forwarded-Host;
        proxy_hide_header X-Forwarded-Server;
        proxy_hide_header X-Forwarded-For;
        proxy_hide_header Referrer-Policy;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-XSS-Protection;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;
        proxy_pass https://jcms.api.gateway/secureCode/v1.0/;
        allow all;
    }

    location /token {
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-Forwarded-Host $host;
        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $http_host;

        proxy_read_timeout 5m;
        proxy_send_timeout 5m;

        proxy_hide_header X-Forwarded-Host;
        proxy_hide_header X-Forwarded-Server;
        proxy_hide_header X-Forwarded-For;
```

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
        proxy_hide_header Referrer-Policy;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-XSS-Protection;
        proxy_hide_header Content-Security-Policy;
        proxy_hide_header Strict-Transport-Security;

        add_header 'Access-Control-Allow-Origin' '*';
        add_header 'Access-Control-Allow-Methods' 'GET, POST, PUT,
PATCH, DELETE, OPTIONS';
        add_header 'Access-Control-Allow-Headers' '*';
        add_header 'Access-Control-Max-Age' 86400;
        proxy_pass https://jcms.token.gateway/oauth2/token;
        allow all;
    }

    location /schedular/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/scheduler/v1.0/;
    }

    location /cookie/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/cookie/v1.0/;
    }

    location /notification/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/notification/v1.0/;
```

"CONFIDENTIAL - FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
    }

    location /grievance/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/grievance/v1.0/;
    }

    location /translator/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/translator/v1.0/;
    }

    location /consent/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/consent/v1.0/;
    }

    location /audit/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/audit/v1.0/;
    }
    location /partnerportal/ {
```

"CONFIDENTIAL - FOR NeGD (MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;
        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;
        proxy_pass https://jcms.api.gateway/partnerportal/v1.0/;
    }

    location /scheduler/ {
        access_by_lua_file
/app/lua/nginx/conf.d/lua/partnerportal.lua;
        include /app/lua/nginx/conf.d/cors-config/preflight.conf;

        proxy_set_header X-User $http_x_user;
        proxy_set_header Authorization $http_authorauthorization;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header Host $host;

        proxy_pass https://jcms.api.gateway/scheduler/v1.0/;
    }

    include /app/lua/nginx/conf.d/locations/user.portal.conf;
    include /app/lua/nginx/conf.d/locations/unsecure.partner.conf;
}
```

Example-Lua use for Partner Portal: partner-portal.lua

```
local http = require "resty.http"
local cjson = require "cjson.safe"
-- get token from incoming headers
local headers = ngx.req.get_headers()
local auth_header = headers["Authorization"] or headers["x-session-
token"] or headers["X-Session-Token"]
local businessId = headers["businessId"] or headers["businessid"] or
headers["business-id"]
if not auth_header then
    return ngx.exit(401)
end
-- extract token
local _, _, token = string.find(auth_header, "Bearer%s+(.+)")
if not token then
    token = auth_header
end
if not token or token == "" then
    return ngx.exit(401)
```

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
end
-- make HTTP request to introspection endpoint
local httpc = http.new()
local body = ngx.encode_args({ token = token })
local res, err =
httpc:request_uri("https://127.0.0.1:8443/oauth2/validate", {
    method = "POST",
    body = body,
    headers = {
        ["Content-Type"] = "application/x-www-form-urlencoded",
    },
    ssl_verify = false
})
if not res then
    ngx.log(ngx.ERR, "introspection failed: ", err)
    return ngx.exit(500)
end
if res.status ~= 200 then
    ngx.header["Access-Control-Allow-Origin"]  = "*"
    ngx.header["Access-Control-Allow-Methods"] = "GET, POST,
OPTIONS"
    ngx.header["Access-Control-Allow-Headers"] = "Authorization,
Content-Type"
    ngx.header["Access-Control-Max-Age"]       = "3600"
    ngx.log(ngx.ERR, "introspection http status: ", res.status)
    return ngx.exit(401)
end
-- parse JSON response
local data = cjson.decode(res.body)
if not data or not data.active then
    return ngx.exit(401)
end
-- optionally pass info upstream
if data.sub then
    ngx.req.set_header("X-User", data.sub)
end
-- ngx.req.set_header("X-Introspection", res.body)
-- set Authorization header for upstream
if data.accessToken and data.accessToken ~= "" then
    ngx.req.set_header("Authorization", "Bearer " ..
data.accessToken)
end
```

Example userportal.lua:

```
local http = require "resty.http"
```

"CONFIDENTIAL - FOR NeGD/MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
local cjson = require "cjson.safe"
local headers = ngx.req.get_headers()
local access_token = headers["x-secure-code"] or
headers["accessToken"]
if not access_token then
    ngx.log(ngx.ERR, "Missing access token in headers")
    return ngx.exit(401)
end

local httpc = http.new()
local res, err =
httpc:request_uri("https://127.0.0.1:8443/secureCode/validate", {
    method = "GET",
    headers = {
        ["accessToken"] = access_token,
    },
    keepalive = false,
    ssl_verify = false
})
if not res then
    ngx.log(ngx.ERR, "Validation request failed: ", err)
    return ngx.exit(500)
end
if res.status ~= 200 then
    ngx.log(ngx.ERR, "Validation API returned status: ", res.status)
    return ngx.exit(401)
end
local data = cjson.decode(res.body)
if not data or not data.active then
    ngx.log(ngx.ERR, "Session inactive or invalid")
    return ngx.exit(401)
end
-- Set headers for backend
ngx.req.set_header("X-Session-Active", "true")
if data.accessToken then
    ngx.req.set_header("Authorization", "Bearer " ..
data.accessToken)
else
    ngx.log(ngx.ERR, "Missing accessToken in validation response")
    return ngx.exit(500)
end
```

a) Example tenant-auth.lua:

```
local http = require "resty.http"
local cjson = require "cjson.safe"
```

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
-- read body (required for POST/PUT)
ngx.req.read_body()
local body = ngx.req.get_body_data()

local identity = nil

if body then
    local json = cjson.decode(body)
    if json and json.spoc then
        -- pick mobile if not empty
        if json.spoc.mobile and json.spoc.mobile ~= "" then
            identity = json.spoc.mobile
        -- else pick email
        elseif json.spoc.email and json.spoc.email ~= "" then
            identity = json.spoc.email
        end
    end
end

-- read secure-code header
local headers = ngx.req.get_headers()

local secret_code = headers["Secret-Code"]
    or headers["secret-code"]
    or headers["SECURE-CODE"]

-- if identity header exists, override body logic
if not identity then
    identity = headers["identity-value"]
        or headers["Identity-Value"]
        or headers["identity-Value"]
end

-- if still not found → unauthorized
if not secret_code or not identity then
    return ngx.exit(401)
end

-- make HTTP request to introspection endpoint
local httpc = http.new()

local res, err =
httpc:request_uri("https://127.0.0.1:8443/secureCode/tenant/validate
", {
    method = "GET",
    headers = {
```

"CONFIDENTIAL - FOR NeGD/(MeitY) USE ONLY"

"This document is submitted only for Code for Consent Challenge, organised by NeGD (a part of MeitY) and shall not be used for any other purpose."

```
        ["secret-code"] = secret_code,
        ["identityvalue"] = identity,
    },
    ssl_verify = false
})

if not res then
    ngx.log(ngx.ERR, "introspection failed: ", err)
    return ngx.exit(500)
end

if res.status ~= 200 then
    ngx.log(ngx.ERR, "introspection status: ", res.status)
    return ngx.exit(401)
end

local data = cjson.decode(res.body)
if not data or not data.active then
    return ngx.exit(401)
end
```

"CONFIDENTIAL – FOR NeGDv(MeitY) USE ONLY"